

ProductionManagementAI — Production Order List Queries — Database Design Document (テーブル定義書)

002_DB — requirements REQ-020–REQ-027 (`work-items/WI-003/brief.md`), basic design 002_BD (`docs/en/010_basic-design/002/002_BD_production-order-list.md`), implements 002_DD and 002_DD-API (`docs/en/020_detailed-design/` , to be written).

Revised by WI-005 (2026-09-23): the seeded notes are Japanese (see the seed table); nothing else changes.

This document adds **no table and no column**. Screen B reads the schema 001_DB defined; what it needs from the database is index support for its filters and default sort, a defined count query, a stable paging order, and demo data. Everything 001_DB already states about `products` , `production_orders` and `production_order_number_counters` stays in force and is not restated here.

Physical naming follows 000_DB and 001_DB: `snake_case` , EF Core `pk_ / fk_ / ix_ / ck_` constraint names.

Table list

Table	Physical name	Purpose	Change in 002_DB
Products	<code>products</code>	Product reference data (001_DB)	none — read only, joined for the product column and filter options
Production orders	<code>production_orders</code>	One row per production order (001_DB)	two new indexes; demo seed rows
Production order number counters	<code>production_order_number_counters</code>	Order-number sequence per year (001_DB)	seed rows only, to stay consistent with the seeded orders

ER diagram and relationships

Unchanged from 001_DB — see its ER diagram. Screen B traverses exactly one relationship:

Table	Related table	Relationship (1:1 / 1:N / N:N)	FK column
<code>products</code>	<code>production_orders</code>	1:N	<code>production_orders.product_id</code>

Table definitions

No column is added, removed or retyped. The columns Screen B reads:

`production_orders` (read projection for SCR-002)

Item name	Physical name	Used for
Id	<code>id</code>	Row link target <code>/production-orders/{id}</code> (REQ-025)
OrderNumber	<code>order_number</code>	Displayed column, sort key, fragment filter, paging tie-breaker (REQ-021–REQ-024)
ProductId	<code>product_id</code>	Product filter; join key for the displayed product (REQ-021, REQ-022)
Quantity	<code>quantity</code>	Displayed column, sort key
DueDate	<code>due_date</code>	Displayed column, default sort key, range filter, overdue marker (REQ-021–REQ-023)
Status	<code>status</code>	Displayed column, sort key, multi-select filter (REQ-021–REQ-023)
UpdatedAtUtc	<code>updated_at_utc</code>	Displayed column, sort key

`notes`, `created_at_utc`, `order_year`, `order_seq` and `xmin` are **not** selected by the list query — the projection is explicit, so a wider row is never fetched than the screen shows. `products.sku` and `products.name` are selected through the join for the product column (002_BD M-06).

Access patterns

The list screen issues exactly two statements per request (FN-010). Both are built by EF Core as parameterized queries with only the *applied* filters composed into them — an unused filter contributes no predicate at all, rather than a `(@p IS NULL OR ...)` clause that would spoil the plan.

Page query

```

SELECT o.id, o.order_number, o.quantity, o.due_date, o.status, o.updated_at_utc,
       p.sku, p.name
FROM production_orders o
JOIN products p ON p.id = o.product_id
WHERE o.status = ANY (@statuses)           -- only when the status filter is set
      AND o.product_id = @productId         -- only when the product filter is set
      AND o.due_date >= @dueFrom            -- only when set (inclusive)
      AND o.due_date <= @dueTo             -- only when set (inclusive)
      AND o.order_number LIKE @pattern ESCAPE '\ ' -- only when the fragment is set
ORDER BY <sort expression> <asc|desc>, o.order_number ASC
LIMIT @pageSize OFFSET (@page - 1) * @pageSize;

```

Count query — the same `WHERE`, `SELECT count(*)`, and **no join**: every filter predicate is on `production_orders`, so the product join is needed only for display and for the product sort key. It supplies the `total` behind 002_BD M-09 and the page count.

Access pattern	Frequency	Served by
First page, no filter, default sort (screen open, the most common request by far)	very high	<code>ix_production_orders_due_date_order_number</code>
Due-date range ± default sort	high	same index (range scan in sort order)
Product filter ± default sort	medium	<code>ix_production_orders_product_id</code> (001_DB, DEC-029) then sort
Order-number fragment	medium	<code>ix_production_orders_order_number_trgm</code>
Status filter	medium	no dedicated index — see "Indexes deliberately not added"
Sort by quantity, status, updated-at or product	low	sort node over the filtered set
Count of matches	once per page request	same predicates as above

Sort-key mapping

API sort value	SQL sort expression	Notes
<code>dueDate</code> (default)	<code>o.due_date</code>	Default direction <code>asc</code> (DEC-002)

API sort value	SQL sort expression	Notes
orderNumber	o.order_number	Lexicographic order equals chronological order within a year, since the format is fixed-width (P0-YYYY-NNNNN)
product	p.sku, p.name	Matches the displayed label order (002_BD M-06)
quantity	o.quantity	
status	CASE o.status WHEN 'Draft' THEN 1 WHEN 'InProgress' THEN 2 WHEN 'Completed' THEN 3 WHEN 'Cancelled' THEN 4 END	Workflow order, not alphabetical — alphabetical would read "Cancelled, Completed, Draft, In progress", which is meaningless to a planner
updatedAt	o.updated_at_utc	

`o.order_number ASC` is appended to every sort (and is the sole key when `sort=orderNumber`). Because it is unique, the total order is deterministic, so paging never repeats or skips a row between pages (REQ-023).

Only these six values reach the query, through an allow-list in the application (002_BD V-13); no client string is ever interpolated into SQL (`ai/rules/database.md`).

Index definitions

Existing indexes from 001_DB are unchanged. New in 002_DB:

Index name	Table	Column(s)	Type	Rationale (access pattern)
ix_production_orders_due_date_order_number	production_orders	due_date, order_number	btree	The screen's default and most frequent query: order by due date with the order-number tie-breaker, optionally restricted to a due-date range. Serves both the ordering and the range predicate from one index, so the common page needs no sort node
ix_production_orders_order_number_trgm	production_orders	order_number (gin_trgm_ops)	GIN (pg_trgm)	The order-number fragment filter is a <i>contains</i> match (REQ-022), which no btree can serve. A trigram index makes <code>LIKE '%...%'</code> index-backed (DEC-010)

`ix_production_orders_order_number_trgm` requires the `pg_trgm` extension, created by the same migration. `pg_trgm` is a trusted extension in PostgreSQL 17, so the migration's owner login can create it without superuser rights, and the runtime login (`pmai_app`) needs no new privilege to use it.

Notes on the trigram index:

- The application upper-cases the fragment and matches with `LIKE` , not `ILIKE` : `order_number` is generated and therefore always upper-case (`P0-YYYY-NNNNN`), so upper-casing the input is enough to make the match case-insensitive, and a plain `LIKE` keeps the index usable without a functional index on

`upper(order_number)` .

- A fragment shorter than three characters yields no full trigram, so PostgreSQL falls back to a scan of the filtered set. That is accepted: 002_BD V-09 bounds the fragment at 20 characters, one- and two-character searches are rare, and the table is small enough that the fallback is cheap. If it ever matters, the UI can require three characters before searching.

Indexes deliberately not added

Candidate	Why not
<code>status</code> (alone, or <code>(status, due_date)</code>)	Only four values, and the two common ones (<code>Draft</code> , <code>InProgress</code>) cover most rows, so a status index is not selective enough to beat a scan of the already-narrow result. A composite <code>(status, due_date)</code> would serve "status filter + default sort", but with <code>status = ANY(...)</code> (multi-select, DEC-005) PostgreSQL must still merge or re-sort across the selected values. Revisit if a status-filtered list becomes the dominant access pattern, or once <code>production_orders</code> exceeds roughly a million rows
<code>(updated_at_utc, order_number)</code> , <code>(quantity, ...)</code> , <code>(status_rank, ...)</code>	One index per sort key would add five indexes that slow every insert and update to serve the screen's least-used orderings. These sorts run as a sort node over the filtered set, which is well within budget at the expected volume (see Performance expectations)
<code>(product_id, due_date, order_number)</code>	Would serve "product filter + default sort" in one scan, but it would also replace the FK index 001_DB kept (DEC-029) and change an index EF Core's convention manages. The product filter alone cuts the set to roughly 1/30 of the table, after which the sort is trivial; not worth changing 001_DB's index set for

Constraints

No constraint is added, dropped or altered. Every constraint in 001_DB stays exactly as defined, including the rules 001_DB lists as application-enforced (due date $\geq$ today, status transitions, the product/quantity lock). Screen B is read-only (DEC-003) and therefore cannot violate any of them.

The overdue marker (002_BD M-08, FN-013) is **not** a database concept: it is computed per row from `due_date` , `status` and the plant-local current date. It is not a stored column and not a filter — the database returns `due_date` and `status` , and the application decides. This keeps "today" out of the schema for the same reason 001_DB gave for the due-date rule: a stored or generated overdue flag would be wrong the moment the date changed.

Transactions and concurrency

Operation	Transaction scope	Concurrency control
List page query + count (FN-010)	No explicit transaction; two statements in one request, each at <code>READ COMMITTED</code>	None needed. The two statements can observe different snapshots, so a row created between them can make the total differ from the page by one. This is normal for a paged list and is accepted rather than fixed with a repeatable-read transaction: the screen is informational, the next query corrects it, and nothing depends on the total being transactionally consistent with the rows

Screen B takes no locks and writes nothing.

Demo seed data

Since **WI-004 (003_DB)**: the 16 seeded `Completed` orders are re-dated and 44 more demo orders are added, so the demo database holds 124 orders (35 `Draft` , 25 `InProgress` , 56 `Completed` , 8 `Cancelled`) and the counter continues after them. The table below describes 002_DB's original 80 rows; 003_DB "Demo seed data" describes the rest.

DEC-007 (user) requires seeded orders so paging, sorting, filtering and the overdue marker are demonstrable and E2E tests have fixtures. 001_DB already seeds 30 products; 002_DB seeds **80 production orders** across them.

Property	Value
Row count	80 (within the 60–100 the user asked for; 4 pages at the default page size of 20, and still more than one page at 50)
Status spread	32 <code>Draft</code> , 24 <code>InProgress</code> , 16 <code>Completed</code> , 8 <code>Cancelled</code> — every filter combination returns a non-empty, differently sized result
Products	Spread across all 30 seeded products, several with more than one order, so the product filter and the product sort are both meaningful
Quantities	1 to roughly 5,000, including boundary values 1 and a large value, so the quantity sort is visibly non-trivial
Due dates	Relative to the plant-local date when the migration runs (DEC-011): 20 in the past (1–14 days ago), 2 today, 58 over the following 40 days. 25 of the 80 orders deliberately share a due date with another, so the order-number tie-breaker is actually exercised; the applied seed produces 55 distinct due dates
Overdue rows	The past-due <code>Draft</code> and <code>InProgress</code> rows — 15 of the 20 past-due rows — carry the overdue marker; the other 5 are <code>Completed</code> / <code>Cancelled</code> and deliberately do not, which demonstrates the second half of M-08
Notes	Present on some rows, absent on others (the list does not show notes; this keeps the seeded rows realistic for Screen A). Seeded as <code>Demo order N</code> ; since WI-005 the migration <code>LocalizeDemoDataToJapanese</code> (001_DB) rewrites them to <code>デモ用の製造指示 N</code>
Order numbers	Issued as <code>P0-<seed year>-00001</code> ... <code>P0-<seed year>-00080</code> , with <code>production_order_number_counters</code> seeded to <code>last_seq = 80</code> for that year so the next order created through Screen A continues the sequence instead of colliding. Verified by an integration test, which asserts the first API-created order is <code>00081</code>

Property	Value
Timestamps	<code>created_at_utc</code> / <code>updated_at_utc</code> staggered over the weeks before the run date, so the "Updated" column and its sort are not all identical

Two properties of this seed are deliberate and worth stating plainly:

- 1. **Due dates are relative to the run date, not fixed calendar dates** (DEC-011). A migration with fixed dates is more reproducible, but every seeded order would be overdue a few months later, and the screen would demo badly forever after. Relative dates keep the mix realistic whenever the stack is rebuilt. Everything else in the seed — ids, order numbers, products, quantities, statuses, the day offsets themselves — is fixed, so the data is deterministic given the run date, and tests assert on counts, statuses and relative dates rather than on absolute dates.
- 2. **The seed only runs into an empty table.** The insert is guarded by `WHERE NOT EXISTS (SELECT 1 FROM production_orders)` , so applying the migration to a database that already holds real orders adds nothing. Re-running or re-applying it is therefore idempotent.

Because the dates are computed at run time, this seed cannot use EF Core `HasData` (which requires constant values, as 001_DB's product seed uses). It is written as an explicit `migrationBuilder.Sql(...)` `INSERT ... SELECT` over a generated series, which is also parameterless and deterministic. The fixed row ids are UUIDv7-shaped constants in the same style as `ProductSeed` .

Application database privileges

No change. Screen B needs only `SELECT` on `production_orders` and `products` , which `pmai_app` already has (001_DB, DEC-016). It needs no privilege on `production_order_number_counters` for this screen, and gains none. Using the `pg_trgm` operator class requires no grant.

The migration that creates the extension, the indexes and the seed runs as the **owner** login, as all migrations do (`deploy/README.md`).

API / DD mapping

DD field names are 002_BD \$3 variable names; API field names are confirmed in 002_DD-API. The list response adds one computed, response-only field with no column.

Table.column	DD field	API field
<code>production_orders.id</code>	(row link target)	<code>id</code>
<code>production_orders.order_number</code>	row order number / <code>orderNumber</code> filter	<code>orderNumber</code> (row), <code>orderNumber</code> (query: fragment)
<code>production_orders.product_id</code>	<code>productId</code> filter	<code>productId</code> (query)

Table.column	DD field	API field
<code>products.sku</code> , <code>products.name</code>	row product label (M-06)	<code>product.sku</code> , <code>product.name</code>
<code>production_orders.quantity</code>	row quantity	<code>quantity</code>
<code>production_orders.due_date</code>	row due date; <code>dueFrom</code> / <code>dueTo</code> filters	<code>dueDate</code> (row, ISO YYYY-MM-DD), <code>dueFrom</code> , <code>dueTo</code> (query)
<code>production_orders.status</code>	row status; <code>status</code> filter	<code>status</code> (row), <code>status</code> (query, repeatable)
<code>production_orders.updated_at_utc</code>	row last updated	<code>updatedAt</code> (ISO 8601 UTC)
— (computed from <code>due_date</code> , <code>status</code> , plant today)	row overdue marker (M-08)	<code>isOverdue</code> (response only)
— (computed by the count query)	result summary (M-09)	<code>total</code> (response only)
— (echoed query controls)	sort, direction, page, page size	<code>sort</code> , <code>dir</code> , <code>page</code> , <code>pageSize</code>
<code>production_orders.notes</code> , <code>created_at_utc</code> , <code>order_year</code> , <code>order_seq</code> , <code>xmin</code>	—	not exposed by the list endpoint

Performance expectations

Sizing assumption, to make the index choices checkable rather than a matter of taste: a single plant creating on the order of 50 production orders a day reaches roughly 15,000 rows a year, so the table stays in the low hundreds of thousands for years. The demo database holds 80.

Query	Expectation
Default page (no filter)	Index scan on <code>ix_production_orders_due_date_order_number</code> , stopping after <code>pageSize</code> rows plus the offset; no sort node
Due-date range + default sort	Same index, bounded range scan
Order-number fragment	Bitmap index scan on the trigram index, then filter
Product filter	Index scan on <code>ix_production_orders_product_id</code> , then a sort of roughly 1/30 of the table
Status filter, or a non-default sort key	Scan of the filtered set plus a sort node — acceptable at the sizes above
Count	Same predicates; an exact <code>count(*)</code> , not an estimate, because the screen reports a precise total (REQ-024)

Revisit this design if the table passes roughly a million rows, if deep paging (a high `page` with a large `pageSize`) becomes common — `OFFSET` grows linearly and keyset paging would then be preferable, at the cost of the "Page n of m" control the screen specifies — or if the status filter becomes the dominant access pattern.

Migration impact and recovery limits

Two migrations, in this order:

1. `AddProductionOrderListIndexes` — indexes and extension.

- **Migration type:** additive. `CREATE EXTENSION IF NOT EXISTS pg_trgm`, then both indexes.
- **Concurrency:** `production_orders` is already in use, so both indexes are created with `CREATE INDEX CONCURRENTLY IF NOT EXISTS` per `ai/rules/database.md`. `CONCURRENTLY` cannot run inside a transaction, and EF Core wraps a migration in one, so these statements are issued with `migrationBuilder.Sql(..., suppressTransaction: true)`. The consequence must be understood before it is run: the migration is **not atomic**. If it fails part-way, PostgreSQL leaves an `INVALID` index behind, which serves no query and must be dropped (`DROP INDEX CONCURRENTLY`) before the migration is retried. Check with `SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;`.
- **Data recovery limit:** none — no data is written or changed.
- **Rollback plan:** `DROP INDEX CONCURRENTLY IF EXISTS` for both indexes; the extension is left in place (dropping it is unnecessary and would break any other object using it). Reversing this migration only makes the list slower, never wrong.

2. `SeedDemoProductionOrders` — demo data.

- **Migration type:** data-inserting, guarded. Inserts 80 `production_orders` rows and one `production_order_number_counters` row, only when `production_orders` is empty.
- **Data recovery limit:** the seed writes only into an empty table, so applying it destroys nothing. Reverting it deletes the 80 seeded rows **by their fixed ids**, which also deletes any edit a user made to a seeded order through Screen A — those edits are not recoverable from the migration. Orders created by users are not touched, because their ids are not in the seeded set.
- **Rollback plan:** `DELETE FROM production_orders WHERE id IN (<the 80 fixed ids>)` and reset that year's counter row. Rows created after the seed are unaffected.
- **Operational note:** because the due dates are computed from the run date, re-applying this migration on a fresh volume at a later date produces different due dates (by design, DEC-011). Test assertions must therefore be written against counts, statuses and offsets from today — never against absolute dates.

Both migrations run as the owner login; neither requires any change to `pmai_app`'s grants.

Both were applied against the local Compose database on 2026-09-22: the two indexes exist, `pg_index` reports no `INVALID` index (so the non-atomic `CONCURRENTLY` path completed), and the table holds the 80 seeded orders across 55 distinct due dates.

The seed also reaches the integration-test database, which runs the same migrations against a throwaway PostgreSQL container. That is deliberate — the list tests need realistic data, and testing the real migration set is the point — but it means order numbers there start after the seeded 80. WI-003 DEC-012 records the

consequence.

Open decisions

Decision	Options	Recommendation	Status
Index for the case-insensitive order-number fragment search	<code>pg_trgm</code> GIN index; btree on <code>upper(order_number)</code> with prefix-only matching; no index	<code>pg_trgm</code> GIN — the only option that keeps REQ-022's "contains" semantics index-backed	decided — <code>work-items/WI-003/decisions.md</code> DEC-010
Seeded due dates: fixed calendar dates or relative to the migration run date	fixed (<code>HasData</code> , fully reproducible); relative (<code>INSERT ... SELECT</code> , realistic forever)	Relative, with everything else fixed and the insert guarded to an empty table	decided — DEC-011
Dedicated index for the status filter	(<code>status</code> , <code>due_date</code>) ; none	None for now; revisit past ~1M rows or if status filtering dominates	decided — recorded here; no separate DEC needed
Keyset paging instead of <code>OFFSET</code>	keyset (fast deep pages, no page numbers); offset (page numbers, linear cost)	Offset — the screen specifies "Page n of m" (REQ-024) and the volume does not justify keyset	decided — recorded here